

Python, on one card

The five moves -- name a value, ask a question, repeat, collect, wrap it in a function -- and the shortcuts that follow from them. Sessions 1 and 4.

The five moves

Everything is a name for a value

```
price_per_item = 249      # int   whole number
gst_rate       = 0.18     # float has a decimal point
top_city       = "Pune"   # str   in quotes
is_open        = True     # bool  True / False

type(gst_rate)           # -> float, when unsure
→ A name is a label, not a box. Reassigning is free: total = total + shipping.
```

Show your work

```
print("price per item:", price_per_item)
print(f"{top_city} sold {units} units") # f-string
print(f"Revenue: Rs {revenue:,.2f}")   # 1,234.50
f"{x:.2f}"    2 decimal places
f"{x:,}"      thousands separator
f"{x:.1%}"    as a percentage
f"{x=}"       prints x=5 (debugging)
```

Ask a question: if / elif / else

```
if order_amount > 1000:
    print("free delivery")
elif order_amount > 500:
    print("half delivery")
else:
    print("delivery charged")

== !=      equal / not equal
> < >= <=  compare
and or not combine
in         is it inside?
```

! A single = assigns a value; == asks a question. The classic first bug.

Do it for every item: for

```
total_sales = 0
for one_order in order_amounts:
    total_sales = total_sales + one_order

for city in ["Pune", "Mumbai"]:
    print(city)

for i in range(5):           # 0 1 2 3 4
    print(i)

range(5)      0 to 4
range(2, 8)   2 to 7
range(0, 10, 2) 0 2 4 6 8
```

Repeat until: while

```
balance = 100000
years   = 0
while balance < target:
    balance = balance * (1 + growth)
    years   = years + 1

! Something inside the loop must move toward the condition, or it never stops. Ctrl-C / stop button.
```

Name a recipe: def

```
def price_with_gst(amount, rate=0.18):
    tax = amount * rate
    return amount + tax

price_with_gst(1000)           # 1180.0
price_with_gst(1000, 0.05)     # 1050.0
→ Write it once, call it everywhere. A function with no return gives you None.
```

Holding many things

List — an ordered row

```
cities = ["Pune", "Mumbai", "Delhi"]
cities[0]      # "Pune"    counting starts at 0
cities[-1]     # "Delhi"   last one
len(cities)    # 3
cities.append("Nagpur")

xs[1:3]        items 1, 2 (stops before 3)
xs[:2]         first two
xs[2:]         from 2 to the end
xs[::-1]       reversed copy
xs.sort()      sort in place
sorted(xs)     sorted copy
```

Dict — a lookup table

```
orders = {"Pune": 3, "Mumbai": 5}
orders["Pune"]      # 3
orders["Goa"] = 1    # add / overwrite
list(orders)         # the keys

for city in orders:
    print(city, orders[city])

d.get(k, 0)          k, or 0 if missing (no crash)
k in d               is the key there?
d.items()            key/value pairs together
d.keys() d.values()  one side only
! d["missing"] raises KeyError. Use .get() when unsure.
```

The four report patterns

```
total = 0                                # running total
for x in amounts:
    total = total + x

count = 0                                # count with a condition
for x in amounts:
    if x > 1000:
        count = count + 1

best = amounts[0]                         # best so far
for x in amounts:
    if x > best:
        best = x

→ Once these are muscle memory, pandas does all three in one line each -- but you will know what it did.
```

Beyond the five moves

Arithmetic, in full

```
+ - * /      / always gives a float: 6/3 -> 2.0
//           divide and throw away the remainder: 7//2 -> 3
%           the remainder: 7%2 -> 1 (even? x%2==0)
**          power: 2**10 -> 1024
+= -= *= x += 1 is x = x + 1

! 6/3 is 2.0, not 2. If you need a whole number, use // or int().
```

Shortcuts for the same patterns

BEYOND CLASS

```
sum(xs)      running total
len(xs)      how many
max(xs) min(xs) best so far
sum(xs)/len(xs) the mean
sorted(xs, reverse=True) biggest first
round(x, 2)   2 decimals
abs(x)        drop the sign
```

List comprehension

BEYOND CLASS

```
big = [x for x in amounts if x > 1000]
with_gst = [x * 1.18 for x in amounts]
names = [c.upper() for c in cities]

→ Read it as: give me EXPR, for each ITEM, if TEST. Same as a for loop with .append(), one line.
```

Loop with the index, or two at once

BEYOND CLASS

```
for i, city in enumerate(cities):
    print(i, city)

for city, n in zip(cities, counts):
    print(city, n)

for city, n in orders.items():
    print(city, n)
```

Sorting the way you want

BEYOND CLASS

```
sorted(orders, key=orders.get, reverse=True) # busiest first
sorted(people, key=len)                     # shortest first

→ key= takes a function; Python calls it on each item and sorts by whatever comes back.
```

Text, cleaned up

BEYOND CLASS

s.strip()	trim spaces at both ends
s.lower() s.upper()	change case
s.title()	Pune Not pune
s.replace(",", " ")	swap text out
s.split(",")	text -> list
", ".join(xs)	list -> text
s.startswith("IN")	test the start
int(s) float(s)	text -> number

When it might fail

BEYOND CLASS

```
try:
    price = float(text)
except ValueError:
    price = 0.0 # a sensible fallback
```

→ Use it at the edges -- reading files, parsing user input. Not as a way to ignore real bugs.

Files, safely

BEYOND CLASS

```
with open("notes.txt") as f:
    text = f.read()
```

```
with open("out.txt", "w") as f:
    f.write("done\n")
```

→ with closes the file for you, even if the code inside raises.

Borrowing other people's work

BEYOND CLASS

```
import math # 3.0
math.sqrt(9)

import numpy as np # the course convention
from pathlib import Path

pip install X # get a library (terminal)
!pip install X # same, inside Colab
python -m venv .venv # a private set of libraries
source .venv/bin/activate # start using it
```

→ Colab already has numpy, pandas, matplotlib and seaborn. On your own machine, make a venv per project so one project cannot break another.

Nothing, and emptiness

BEYOND CLASS

```
answer = None # "no value yet"
if answer is None: ... # is, not ==
```

→ Empty things are False: 0, "", [], {}, None. Anything else is True. So if cities: reads as "if there are any cities".

Two more containers

BEYOND CLASS

```
point = (12.9, 77.6) # tuple: a fixed pair
lat, lon = point # unpack it

seen = set(cities) # set: no duplicates
seen.add("Pune")
len(set(cities)) # how many DISTINCT cities
```

→ A tuple is a list that cannot be edited -- good for things that belong together, like a coordinate.

Two names, one list

BEYOND CLASS

```
b = a # NOT a copy -- both names, same list
b.append(4) # a changed too
```

```
b = a.copy() # a real copy
```

! Lists and dicts are shared by name. Copy before you edit, or the original changes under you.

When it goes wrong

Read the error from the bottom up

NameError	typo, or you never defined it
TypeError	"3" + 4 -- text meets number
ValueError	right type, impossible value
IndexError	list has 3 items, you asked for [5]
KeyError	that key is not in the dict
ZeroDivisionError	you divided by zero
IndentationError	spacing inside a block is off

→ The last line names the problem. The line above it points at your code.

Mistakes everyone makes once

! Counting starts at 0, so the 3rd item is xs[2].

! xs[1:3] stops BEFORE 3 -- you get 2 items, not 3.

! A single = assigns, == compares.

! Blocks are defined by indentation. Be consistent: 4 spaces.

! xs.sort() returns None -- it sorts in place. Use sorted(xs) if you want a copy back.

! Integer vs text: input() and CSV columns give you text. Convert before doing maths.

! 0.1 + 0.2 does not equal 0.3. Never test decimals with ==; use round(a - b, 9) == 0 or math.isclose(a, b).

! Never write def f(xs=[]) -- the empty list is made ONCE and shared by every call. Use xs=None, then if xs is None: xs = [].

! Do not add to or remove from a list while looping over it. Build a new list instead.

Finding out for yourself

```
type(x) # what is this thing?
len(x) # how big is it?
help(round) # what does it take?
dir(x) # what can it do?
```

→ These four answer most questions faster than a search does.

Debugging without any tools

```
print("before loop:", total)
print("x is", x, "of type", type(x))
```

→ Print the thing just before the line that broke. Nine bugs in ten are a value you assumed but never looked at.

→ Then check: is it the type you expected? the length you expected? Fix the assumption, not the symptom.

Living in a notebook

Shift + Enter run the cell, go to the next

Ctrl + Enter run it, stay put

round? what does this do?

%timeit f() how slow is it?

Restart & run all the honest test

! Cells remember what you ran, not what you see. If results stop making sense, restart and run all.

Habits that make code readable

→ Name things in full: total_sales, not ts.

→ A comment says WHY, the code already says what.

→ Four spaces per indent level, always.

→ If you copy a block three times, make it a function.